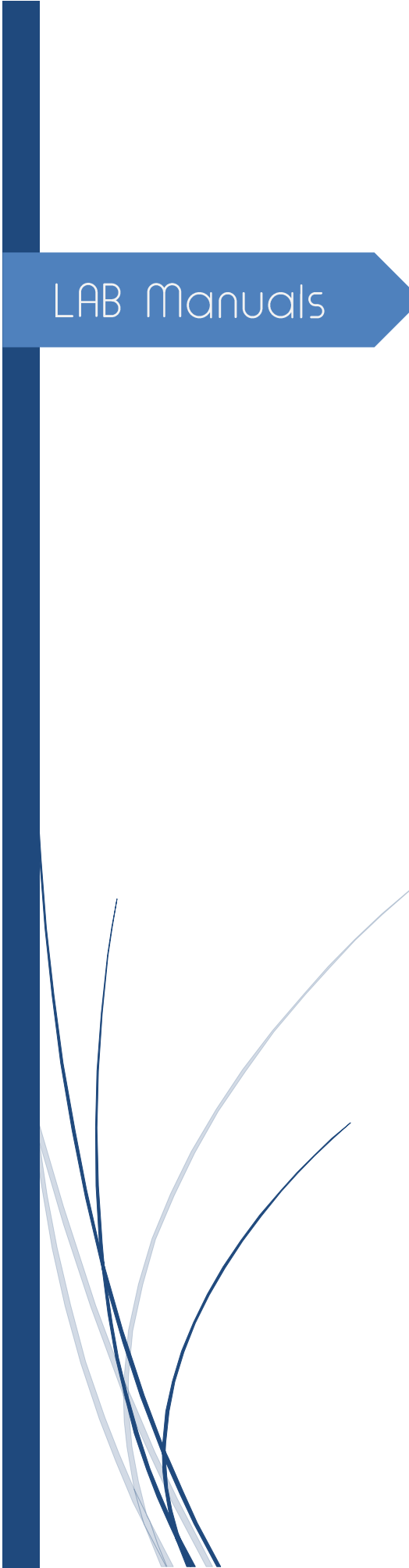




LAB Manuals

Programming Fundamentals

Abdur Rehman

2026(S)-CYS-26



Submitted By:

Abdur Rehman – 2026(S)-CYS-26

(Section: A)

Submitted To:

Hafiz Muhammad Mubashar

For the fulfillment of,

Programming Fundamentals LAB Manuals

Department of Computer Engineering

University of Engineering and Technology, Lahore

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 01	CLOs to be covered: CLO 01
Lab Title: Input/Output, Data Types, Operators, If/Else	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 01

Lab Goals/Objectives:

- Learn the basics of Python programming.
- Understand different data types and variables.
- Perform arithmetic and logical operations using operators.
- Accept input from the user and display output.
- Implement decision-making using if, elif, and else statements.

Equipment Required:

- Computer/Laptop

- Python 3.x
- Visual Studio Code (VS Code)

Theory:

Python is a high-level programming language that is easy to learn and use. Programs interact with users through input and output statements. Variables are used to store data such as integers, floating-point numbers, strings, and Boolean values. Operators perform calculations and comparisons on data. Conditional statements (if, elif, and else) allow the program to make decisions by executing different blocks of code based on specified conditions.

Lab Work:

Task 1:

String output formatting.

Result:

```
12
13 print("Name: Abdur Rehman")
14 print("Department: Computer Engineering")
15 print("University: UET Lahore")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\Microsoft VS Code> & "C:\Users\WAQAS COMPUTER\
Name: Abdur Rehman
Department: Computer Engineering
University: UET Lahore
PS E:\Microsoft VS Code> █
```

Task 2:

Join multiple strings together using a hyphen separator.

Result:

```
11
12 print("UET", "Lahore", "Computer", "Engineering", sep = "-")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab\PF-Lab_Week_01_CYS-26> & "C:\Users\WAQAS COMPUTER\AppData
_01_CYS-26/Task # 02 (Week_01).py"
UET-Lahore-Computer-Engineering
PS E:\PF & DS Lab\PF-Lab_Week_01_CYS-26> █
```

Task 3:

Use newline characters (`\n`) within a single string to format and print multiple pieces of information on separate lines.

Result:

```
12
13 print("Name: Abdur Rehman\nAge: 19\nCity: Lodhran")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab\PF-Lab_Week_01_CYS-26> & "C:\Users\WAQAS COMP
_01_CYS-26/Task # 03 (Week_01).py"
Name: Abdur Rehman
Age: 19
City: Lodhran
PS E:\PF & DS Lab\PF-Lab_Week_01_CYS-26> █
```

Task 4:

Capture user input, convert the age to an integer, and print both values on separate lines.

Result:

```
9 name = input("Enter your name: ")
10 age = int(input("Enter your age: "))
11 print("Your Name: ", name)
12 print("Your Age: ", age)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab\PF-Lab_Week_01_CYS-26> & "C:\Use
_01_CYS-26/Task # 04 (Week_01).py"
Enter your name: Ab Rehman
Enter your age: 19
Your Name: Ab Rehman
Your Age: 19
PS E:\PF & DS Lab\PF-Lab_Week_01_CYS-26> █
```

Task 5:

Declare variables of different data types and print both their values, data types, and the length of the string variable.

Result:

```
1 x = 10
2 y = 3.14
3 z = True
4 name = "UET"
5 print(x)
6 print(y)
7 print(z)
8 print(name)
9 print(type(x))
10 print(type(y))
11 print(type(z))
12 print(type(name))
13 print(len(name))
```

PROBLEMS OUTPUT DEBUG CONSOLE

```
PS E:\PF & DS Lab\PF-Lab_Week_01_01_CYS-26/Task # 05 (Week_01).py
10
3.14
True
UET
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'str'>
3
PS E:\PF & DS Lab\PF-Lab_Week_01_01_CYS-26/Task # 05 (Week_01).py
```

Task 6:

Use of Conditional Statement (If/Else).

Result:

```
8
9 age = int(input("Enter your age: "))
10 if age >= 18:
11     print("You are eligible to vote.")
12 else:
13     print("You are not eligible to vote.")
14
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local\Microsoft\Windows\Terminal\Profiles\Task # 01 (Week_02).py"
Enter your age: 19
You are eligible to vote.
PS E:\PF & DS Lab> 
```

Conclusions:

In this lab, the basic concepts of Python programming were learned. User input, output statements, variables, data types, operators, and conditional statements were successfully implemented. This lab established a strong foundation for writing simple Python programs.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 02	CLOs to be covered: CLO 01
Lab Title: For Loops	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 02

Lab Goals/Objectives:

- Understand the concept of repetition in programming.
- Learn to use the for loop for iterative tasks.
- Generate sequences using the range() function.
- Solve problems involving repeated execution of statements.

Equipment Required:

- Computer/Laptop
- Python 3.x
- Visual Studio Code (VS Code)

Theory:

A loop is used to execute a block of code multiple times. The for loop is commonly used when the number of iterations is known. In Python, the range() function generates a sequence of numbers that controls loop execution. For loops simplify repetitive tasks such as displaying patterns, calculating sums, and processing collections of data.

Lab Work:

Task 1:

Use of For Loop (along with conditional statement).

Result:

```
1 sentence = input("Enter a sentence: ")
2 vowels = 0
3 consonants = 0
4 for ch in sentence.lower():
5     if ch in "aeiou":
6         vowels += 1
7     elif ch.isalpha():
8         consonants += 1
9 print("Number of vowels:", vowels)
10 print("Number of consonants:", consonants)
11
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local\Microsoft\Windows\Terminal\02 (Week_02).py"
Enter a sentence: My name is Ab Rehman.
Number of vowels: 6
Number of consonants: 10
PS E:\PF & DS Lab>
```

Task 2:

Build a pool of allowed characters based on user preferences and randomly select from it to generate a password.

Result:

```
Week_02).py"
```

```
Enter your password length: 6
```

```
Include uppercase letters? (yes/no): yes
```

```
Include lowercase letters? (yes/no): yes
```

```
Include digits? (yes/no): no
```

```
Include special characters? (yes/no): yes
```

```
Your password is: Qqf!BE
```

```
PS E:\PE & DS Lab>
```

Conclusions:

In this lab, the use of the **for loop** was studied to perform repetitive tasks efficiently. Different looping problems were solved using the **range()** function, improving the understanding of iteration and code optimization

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 03	CLOs to be covered: CLO 01
Lab Title: While Loops	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 03

Lab Goals/Objectives:

- Learn how while loops work.
- Understand condition-controlled iteration.
- Develop programs that repeat until a specified condition becomes false.
- Compare while loops with for loops.

Equipment Required:

- Computer/Laptop
- Python 3.x
- Visual Studio Code (VS Code)

Theory:

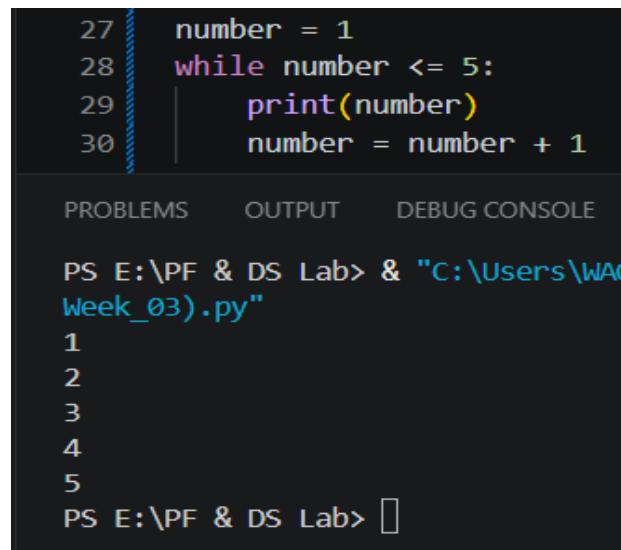
A while loop repeatedly executes a block of code as long as a given condition remains true. It is useful when the number of iterations is unknown beforehand. Care must be taken to update the loop condition; otherwise, the program may enter an infinite loop. While loops are commonly used for user input validation and menu-driven programs.

Lab Work:

Task 1:

Use a variable that starts at 1 and increases by 1 each time until it hits 5 to print numbers in order.

Result:



```
27 | number = 1
28 | while number <= 5:
29 |     print(number)
30 |     number = number + 1
```

PROBLEMS OUTPUT DEBUG CONSOLE

```
PS E:\PF & DS Lab> & "C:\Users\WAG
Week_03).py"
1
2
3
4
5
PS E:\PF & DS Lab> 
```

Task 2:

Repeatedly ask for numbers and add them to a running total until the user enters 0 to finish.

Result:

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 04	CLOs to be covered: CLO 01
Lab Title: Nested Loops	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 04

Lab Goals/Objectives:

- Learn the concept of nested loops.
- Develop pattern-printing programs.
- Solve problems involving multiple levels of repetition.
- Understand how inner and outer loops interact.

Equipment Required:

- Computer/Laptop
- Python 3.x
- Visual Studio Code (VS Code)

Theory:

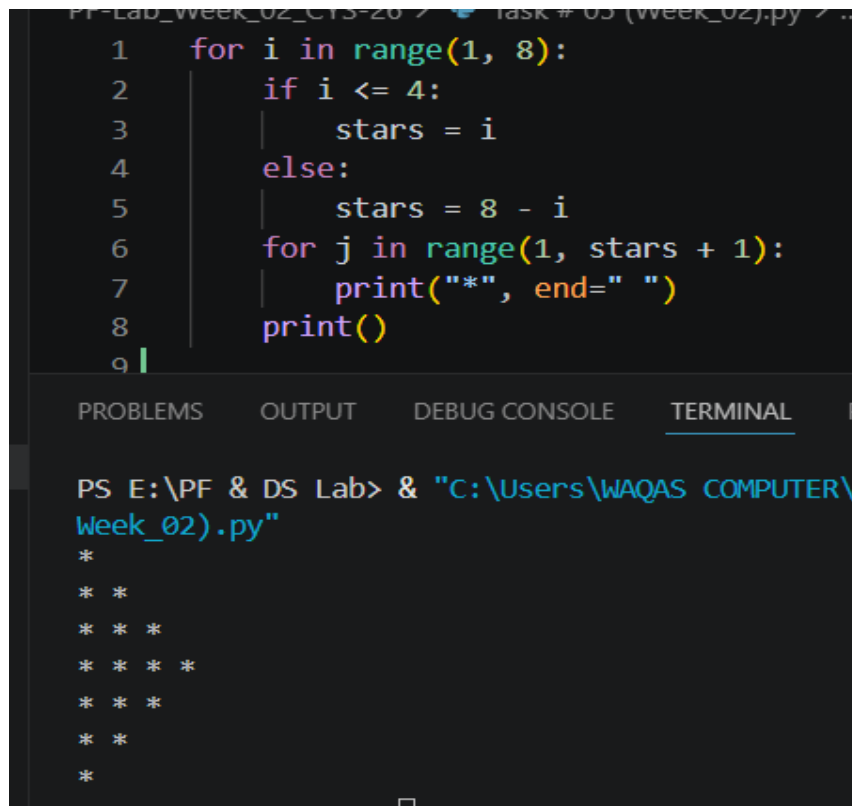
A nested loop is a loop placed inside another loop. The outer loop controls how many times the inner loop executes. Nested loops are useful for creating patterns, working with tables, and solving problems involving two-dimensional data structures.

Lab Work:

Task 1:

A nested loop strategy that dynamically calculates star counts to print an increasing and then decreasing pattern.

Result:



```
1  for i in range(1, 8):
2      if i <= 4:
3          stars = i
4      else:
5          stars = 8 - i
6      for j in range(1, stars + 1):
7          print("*", end=" ")
8      print()
9
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\Week_02).py"
*
* *
* * *
* * * *
* * *
* *
*
```

Task 2:

A nested loop that tests every number in a user-defined range for exactly two divisors to identify, print, and sum the prime numbers.

Result:

```
2  start = int(input("Enter starting number: "))
3  end = int(input("Enter ending number: "))
4  sum_of_primes = 0
5  for num in range(start, end + 1):
6      if num > 1:
7          count = 0
8          for i in range(1, num + 1):
9              if num % i == 0:
10                 count += 1
11             if count == 2:
12                 print(num)
13                 sum_of_primes += num
14  print("Sum of prime numbers:", sum_of_primes)
15
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Loc
Week_02).py"
Enter starting number: 1
Enter ending number: 10
2
3
5
7
Sum of prime numbers: 17
PS E:\PF & DS Lab> 
```

Conclusions:

In this lab, nested loops were used to solve pattern-printing and repetitive programming problems. The relationship between outer and inner loops became clear, improving logical thinking and problem-solving skills.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 05	CLOs to be covered: CLO 02
Lab Title: Functions	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 05

Lab Goals/Objectives:

- Understand the purpose of functions.
- Learn to define and call user-defined functions.
- Pass arguments and return values.
- Improve program organization through modular programming.

Equipment Required:

- Computer/Laptop
- Python 3.x

- Visual Studio Code (VS Code)

Theory:

A function is a reusable block of code that performs a specific task. Functions reduce code duplication and improve readability. Python provides built-in functions as well as user-defined functions created using the `def` keyword. Functions may accept parameters and return values using the `return` statement.

Lab Work:

Task 1:

Create a reusable factorial function using a loop, then call it to calculate and display both permutation and combination formulas based on user inputs.

Result:

```
Week_03.py
Enter a number: 6
The factorial of 6 is 720
Enter the value of n: 5
Enter the value of r: 4
Permutation (nPr)= 120
Combination (nCr)= 5
PS E:\PF & DS Lab>
```

Task 2:

Use a lambda function to find the larger of two inputs.

Result:

```
10 num1 = int(input("Enter the first number: "))
11 num2 = int(input("Enter the second number: "))
12 large = lambda a, b: a if a > b else b
13 result = large(num1, num2)
14 print("Large number is: ", result)
15 # n = int(input("Enter the range for table: "))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local\Microsoft\Windows\apps\python\python.exe" "C:\Users\WAQAS COMPUTER\AppData\Local\Microsoft\Windows\apps\python\python.exe" Week_03.py
Enter the first number: 45
Enter the second number: 43
Large number is: 45
```

Task 3:

Define separate conversion functions for temperature scales and use conditional logic to execute the user's selected choice.

Result:

```
Week_03.py
1. Fahrenheit to Celsius
2. Celsius to Fahrenheit
Enter your choice (1 or 2): 2
Enter temperature in Celsius: 37
Temperature in Fahrenheit is: 98.6
PS E:\PF & DS Lab>
```

Task 4:

Define arithmetic functions that return calculation results, and use conditional statements to trigger the matching operation based on user input.

Result:

```
Menu
1. Add
2. Subtract
3. Multiply
4. Divide
Enter your choice (1-4): 3
Enter first number: 15
Enter second number: 12
Result: 180.0
PS E:\PF & DS Lab> & "C:\Users\W
Week_04).py"
Menu
1. Add
2. Subtract
3. Multiply
4. Divide
Enter your choice (1-4): 1
Enter first number: 56
Enter second number: 23
Result: 79.0
```

Conclusions:

In this lab, user-defined functions were created to organize programs into reusable modules. The use of parameters and return values improved program structure and reduced code repetition.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 06	CLOs to be covered: CLO 02
Lab Title: Local and Global Variables	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 06

Lab Goals/Objectives:

- Differentiate between local and global variables.
- Understand variable scope.
- Learn how Python accesses variables in different scopes.
- Use the global keyword when required.

Equipment Required:

- Computer/Laptop
- Python 3.x
- Visual Studio Code (VS Code)

Theory:

Variables declared inside a function are local variables and can only be accessed within that function. Variables declared outside all functions are global variables and are accessible throughout the program. Python follows scope rules to determine which variable should be used during execution.

Lab Work:

Task 1:

Create a local variable inside a function that is isolated and only accessible within that specific function block.

Result:

```
15 def print_score():  
16     score = 100  
17     print("Inside function score is:", score)  
18     print_score()  
19
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local\Microsoft\Windows\Terminal\Sessions\Week_09).py"  
Inside function score is: 100  
PS E:\PF & DS Lab>
```

Task 2:

Define a global variable outside of all functions so that any function can read its value.

Result:

```
15 total_money = 500
16 def check_wallet():
17     print("Your total money is:", total_money)
18     check_wallet()
19
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local\Programs\Python\Python39-64\python.exe" C:\Users\WAQAS COMPUTER\AppData\Local\Programs\Python\Python39-64\Scripts\pip.exe install Week_09).py"
Your total money is: 500
```

Conclusions:

In this lab, the difference between local and global variables was understood. The concept of variable scope was explored, enabling better management of variables in larger programs.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 07	CLOs to be covered: CLO 03
Lab Title: List, Tuples	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 07

Lab Goals/Objectives:

- Learn to store multiple values in a single variable.
- Perform operations on lists and tuples.
- Understand the differences between mutable and immutable data structures.
- Access and manipulate elements using indexing and slicing.

Equipment Required:

- Computer/Laptop
- Python 3.x

- Visual Studio Code (VS Code)

Theory:

Lists and tuples are collection data types in Python. Lists are mutable, allowing elements to be added, removed, or modified. Tuples are immutable, meaning their contents cannot be changed after creation. Both support indexing, slicing, and iteration, making them useful for managing collections of data.

Lab Work:

Task 1:

Create a mutable list of items that allows you to dynamically add new products using the `append()` method.

Result:

```
18 cart = ["laptop", "mouse", "keyboard"]
19 cart.append("monitor")
20 print(cart)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local\Microsoft\Windows\Terminal\
Week_02).py"
['laptop', 'mouse', 'keyboard', 'monitor']
PS E:\PF & DS Lab>
```

Task 2:

Store numerical data in a mutable list so you can easily update individual values using index numbers.

Result:

```
18 scores = [85, 92, 78]
19 scores[0] = 88
20 print(scores)
```

PROBLEMS OUTPUT DEBUG CONSOLE

```
PS E:\PF & DS Lab> & "C:\Users\W
Week_02).py"
[88, 92, 78]
PS E:\PF & DS Lab>
```

Task 3:

Lock a user's fixed profile username and password securely together within a single, unchangeable tuple.

Result:

```
18 user_credentials = ("AbR", "1044")
19 print("Logged in user:", user_credentials)
20
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData
Week_02).py"
Logged in user: ('AbR', '1044')
PS E:\PF & DS Lab>
```

Conclusions:

In this lab, lists and tuples were used to store and manage collections of data. Their similarities and differences were understood, along with operations such as indexing, slicing, and element access.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 08	CLOs to be covered: CLO 03
Lab Title: Sets, Dictionary	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 08

Lab Goals/Objectives:

- Understand the use of sets and dictionaries.
- Perform common set operations.
- Store and retrieve data using key-value pairs.
- Compare different Python collection types.

Equipment Required:

- Computer/Laptop
- Python 3.x
- Visual Studio Code (VS Code)

Theory:

A set is an unordered collection of unique elements used to eliminate duplicates and perform mathematical set operations. A dictionary stores data as key-value pairs, allowing efficient access and modification of information using keys. These data structures are widely used for organizing and processing data.

Lab Work:

Task 1:

The strategy is to validate marks for a grading scale and use a loop to dynamically populate a student records dictionary with list values using the `setdefault()` method.

Result:

```
Obtained Marks: 250
Total Marks: 300
83.33333333333334
B+
Enter number of students: 2
Enter student's name: A
Enter student's marks: 245
Enter student's name: B
Enter student's marks: 255
Enter student's name: C
Enter student's marks: 250
{'A': ['245'], 'B': ['255'], 'C': ['250']}
PS F:\DE & DS Lab>
```

Task 2:

Store botanical and common names as pairs inside a dictionary, retrieve a specific value using its exact key, and use a loop to print all records.

Result:

```
9  fruits = {
10      "Malus domestica": "Apple",
11      "Musa acuminata": "Banana",
12      "Vitis vinefera": "Grapes",
13      "Mangifera indica": "Mango"
14  }
15  for key, values in fruits.items():
16      print(f"{key} : {values}")
17  print(fruits["Mangifera indica"])
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\OneDrive\Desktop\Week_10\Week_10).py"
Malus domestica : Apple
Musa acuminata : Banana
Vitis vinefera : Grapes
Mangifera indica : Mango
Mango
PS E:\PF & DS Lab>
```

Task 3:

Set (using built in function).

Result:

```
18  s1 = {1,2,3,}
19  s2 = {"A, B, C"}
20  s3 = {7, "Abid", "4, (variable) s3"}
21  a = set.union(s1,s2,s3)
22  print(a)
23
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\OneDrive\Desktop\Week_11\Week_11).py"
{1, 2, 3, '4, 5, 6', 7, 'A, B, C', 'Abid'}
PS E:\PF & DS Lab>
```

Conclusions:

In this lab, sets and dictionaries were used to organize data efficiently. Set operations and dictionary key-value pairs were practiced, providing a better understanding of Python's collection data types.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 10	CLOs to be covered: CLO 02
Lab Title: Recursion	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 10

Lab Goals/Objectives:

- Understand recursive problem solving.
- Learn to write recursive functions.
- Identify base and recursive cases.
- Compare recursion with iteration.

Equipment Required:

- Computer/Laptop
- Python 3.x
- Visual Studio Code (VS Code)

Theory:

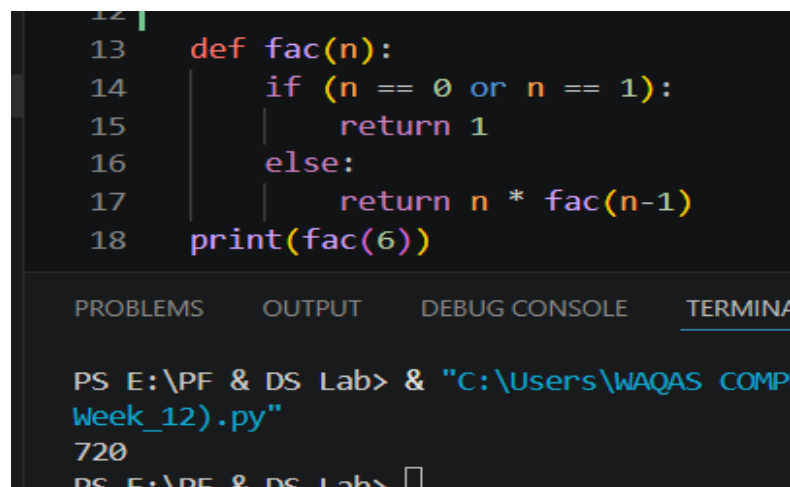
Recursion is a programming technique in which a function calls itself to solve a problem. Every recursive function must have a base case to stop further recursive calls and a recursive case that reduces the problem into smaller sub-problems. Recursion is commonly used in mathematical calculations and tree-based algorithms.

Lab Work:

Task 1:

A recursion strategy where a function solves a problem by continually calling itself with a smaller input until it reaches a base condition.

Result:



```
12 |
13 | def fac(n):
14 |     if (n == 0 or n == 1):
15 |         return 1
16 |     else:
17 |         return n * fac(n-1)
18 | print(fac(6))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMP\OneDrive\Desktop\Week_12\Week_12.py"
720
PS E:\PF & DS Lab> █
```

Task 2:

Find a specific sequence number using recursion by summing the outputs of two self-referencing function calls until hitting the base conditions.

Result:

```
11 |  
12 ∨ def fibonacci(n):  
13 ∨     if n == 0:  
14         return 0  
15 ∨     if n == 1:  
16         return 1  
17         return fibonacci(n - 1) + fibonacci(n - 2)  
18 print(f"The 8th Fibonacci number is: {fibonacci(8)}")  
19
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local\Programs\Python\Python39-64\python.exe" C:\Users\WAQAS COMPUTER\AppData\Local\Programs\Python\Python39-64\Scripts\python.exe Week_12).py"  
The 8th Fibonacci number is: 21  
PS E:\PF & DS Lab>
```

Task 3:

Compress a recursion algorithm into a single line using a ternary operator that multiplies the current number by a self-referencing call until reaching the base condition.

Result:

```
15 |
16 | def fac(n):
17 |     return 1 if n <= 1 else n * fac(n - 1)
18 | print(fac(4))
19 |
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local\Microsoft\Windows\Terminal\
Week_12).py"
24
PS E:\PF & DS Lab> 
```

Conclusions:

In this lab, recursive functions were implemented to solve problems by dividing them into smaller sub-problems. The importance of base cases and recursive calls was clearly understood.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 11	CLOs to be covered: CLO 04
Lab Title: Exception Handling	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 11

Lab Goals/Objectives:

- Understand runtime errors and exceptions.
- Learn to use try, except, else, and finally blocks.
- Develop robust programs that handle unexpected situations.
- Prevent program crashes caused by invalid input.

Equipment Required:

- Computer/Laptop
- Python 3.x
- Visual Studio Code (VS Code)

Theory:

Exceptions are runtime errors that occur during program execution. Python provides exception handling mechanisms using try and except blocks to catch and manage errors gracefully. Optional else and finally blocks provide additional control over program execution and cleanup activities.

Lab Work:

Task 1:

Use an exception handling block to catch a division error and display a friendly message instead of letting the program crash.

Result:

```
15 try:
16     result = 10 / 0
17 except:
18     print("You cannot divide a number by zero!")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local
Week_13).py"
You cannot divide a number by zero!
PS E:\PF & DS Lab> 
```

Task 2:

Use an exception handling block to catch invalid data conversion errors and print a friendly message if the user enters non-numeric text instead of an integer.

Result:

```
9 try:
10     print(int(input("Enter a number: ")))
11 except:
12     print("Hi there! How are you?")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\
Week_13).py"
Enter a number: Hello
Hi there! How are you?
PS E:\PF & DS Lab> 
```

Task 3:

Use an exception handling block to intercept invalid integer input and print a fallback text message instead of letting the script crash.

Result:

```
9      try:  
10         a = int(input("Enter any number: "))  
11         print(a)  
12     except:  
13         print("Hello World")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData  
Week_13).py"  
Enter any number: ABC  
Hello World  
PS E:\PF & DS Lab>
```

Conclusions:

In this lab, exception handling techniques were used to prevent program crashes caused by runtime errors. The use of **try**, **except**, **else**, and **finally** blocks improved the reliability of Python programs.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 12	CLOs to be covered: CLO 02
Lab Title: Lambda Functions	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 12

Lab Goals/Objectives:

- Learn anonymous function creation.
- Understand the syntax of lambda expressions.
- Use lambda functions for simple operations.
- Improve code readability for short functions.

Equipment Required:

- Computer/Laptop
- Python 3.x
- Visual Studio Code (VS Code)

Theory:

A lambda function is a small anonymous function defined using the lambda keyword. It can accept multiple arguments but contains only one expression. Lambda functions are commonly used where a simple function is needed for a short period, especially with higher-order functions.

Lab Work:

Task 1:

Define an anonymous lambda function to calculate and print the square of a single number.

Result:

```
5 a = lambda x: x * x
6 print(a(3))
7
```

PROBLEMS OUTPUT DEBUG CONSOLE

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS C"
y"
9
```

Task 2:

An anonymous lambda function that accepts two inputs, calculates their sum, and prints the result.

Result:

```
8 add = lambda a, b: a + b
9 print(add(5, 3))
10
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS C"
y"
8
PS E:\PF & DS Lab>
```

Conclusions:

In this lab, lambda functions were used to write short and efficient anonymous functions. Their application in simplifying simple programming tasks was successfully understood.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 13	CLOs to be covered: CLO 02
Lab Title: Map, Filter and Reduce	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 13

Lab Goals/Objectives:

- Learn functional programming techniques.
- Apply map(), filter(), and reduce() functions.
- Process collections efficiently.
- Write concise and readable programs.

Equipment Required:

- Computer/Laptop
- Python 3.x
- Visual Studio Code (VS Code)

Theory:

The `map()` function applies a specified function to every item in an iterable. The `filter()` function selects elements that satisfy a given condition. The `reduce()` function repeatedly combines elements into a single result using a specified function. These functions simplify data processing and improve code efficiency.

Lab Work:

Task 1:

Use the `map()` built-in function to apply a lambda function to every element in a list, transforming the numbers into their squares.

Result:

```
11 a = [1, 2, 3, 4]
12 result = list(map(lambda x: x * x, a))
13 print(result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local\Microsoft\Windows\Terminal\
y"
[1, 4, 9, 16]
PS E:\PF & DS Lab> |
```

Task 2:

Use the `filter()` built-in function to test every element in a list against a condition, keeping only the items that return true

Result:

```
16 ages = [12, 18, 25, 15, 30]
17 adults = list(filter(lambda x: x >= 18, ages))
18 print(adults)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local\Microsoft\Windows\Terminal\
y"
[18, 25, 30]
PS E:\PF & DS Lab> |
```

Task 3:

Import and use the `reduce()` module function to cumulatively apply a calculation across a list from left to right, shrinking it down to a single final value

Result:

```
16 from functools import reduce
17 numbers = [1, 2, 3, 4]
18 total_sum = reduce(lambda x, y: x + y, numbers)
19 print(total_sum)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTER\AppData\Local\
y"
10
PS E:\PF & DS Lab> 
```

Conclusions:

In this lab, the **`map()`**, **`filter()`**, and **`reduce()`** functions were used to process data efficiently. These functional programming techniques helped produce cleaner and more concise code.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 14	CLOs to be covered: CLO 02
Lab Title: OS Library, if name == "main", is vs ==	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 14

Lab Goals/Objectives:

- Learn basic file and directory operations using the OS library.
- Understand the purpose of the `if name == "_main_"` statement.
- Differentiate between `is` and `==` operators.
- Write more organized and reusable Python programs.

Equipment Required:

- Computer/Laptop
- Python 3.x

- Visual Studio Code (VS Code)

Theory:

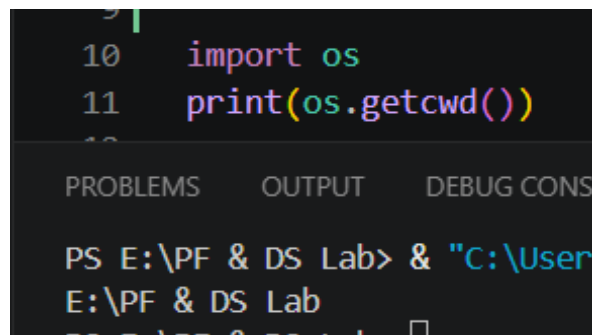
The OS library provides functions for interacting with the operating system, such as managing files and directories. The `if __name__ == "__main__"` statement ensures that specific code executes only when the file is run directly, not when imported as a module. The `==` operator compares values, while the `is` operator checks whether two variables refer to the same object in memory.

Lab Work:

Task 1:

Import the OS Module to interact with the operating system and print the current working directory path.

Result:



```
10 import os
11 print(os.getcwd())
```

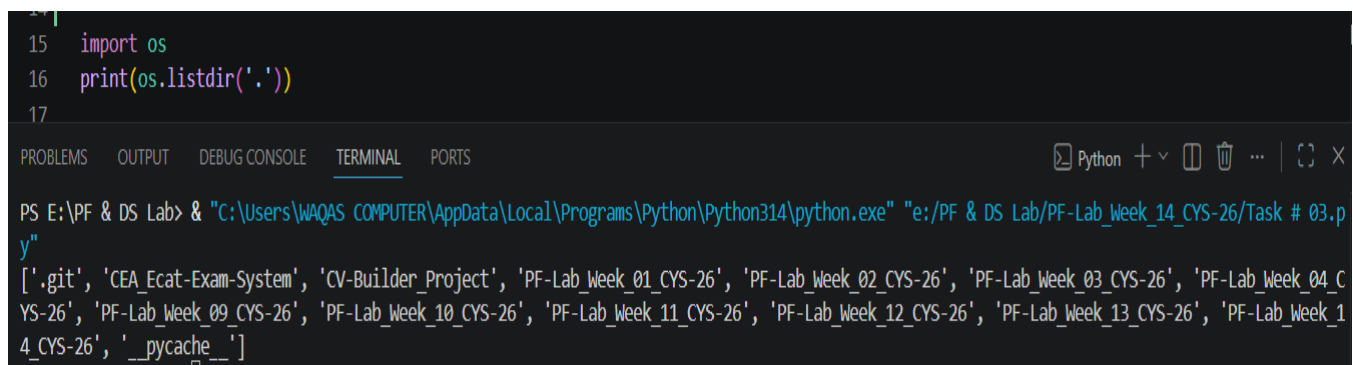
Below the code editor, the terminal output shows the command being executed and the resulting path:

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS\OneDrive\Desktop\Python\Python314\python.exe" "E:\PF & DS Lab\PF-Lab_Week_14_CYS-26\Task # 03.py"
```

Task 2:

Import the OS Module to interact with the operating system and print the contents of current directory.

Result:



```
15 import os
16 print(os.listdir('.'))
```

The terminal output shows the command being executed and the resulting list of directory contents:

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS\OneDrive\Desktop\Python\Python314\python.exe" "E:\PF & DS Lab\PF-Lab_Week_14_CYS-26\Task # 03.py"
```

```
['.git', 'CEA_Ecat-Exam-System', 'CV-Builder_Project', 'PF-Lab_Week_01_CYS-26', 'PF-Lab_Week_02_CYS-26', 'PF-Lab_Week_03_CYS-26', 'PF-Lab_Week_04_CYS-26', 'PF-Lab_Week_09_CYS-26', 'PF-Lab_Week_10_CYS-26', 'PF-Lab_Week_11_CYS-26', 'PF-Lab_Week_12_CYS-26', 'PF-Lab_Week_13_CYS-26', 'PF-Lab_Week_14_CYS-26', '__pycache__']
```

Task 3:

Use the conditional name block script check to ensure the main function executes only when the file is run directly rather than imported.

Result:

```
17
18 def main():
19     print("Running directly")
20 if __name__ == "__main__":
21     main()
22
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
PS E:\PF & DS Lab> & "C:\Users\WAQAS COMPUTE
y"
Running directly
PS E:\PF & DS Lab> 
```

Conclusions:

In this lab, basic operating system functions were explored using the OS library. The purpose of the `if __name__ == "__main__":` statement and the difference between the `is` and `==` operators were clearly understood, leading to better coding practices.

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Programming Fundamentals	Course Code: CMPE-112L
Assignment Type: Lab	Dated:
Semester: 1 st	Session: Spring 2026
Lab/Project/Assignment #: 15	CLOs to be covered: CLO 03
Lab Title: GUI using Qt Designer	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No: 15

Lab Goals/Objectives:

- Understand graphical user interface (GUI) development.
- Design application windows using Qt Designer.
- Connect GUI components with Python code.
- Develop user-friendly desktop applications.

Equipment Required:

- Computer/Laptop
- Python 3.x
- Visual Studio Code (VS Code)

Theory:

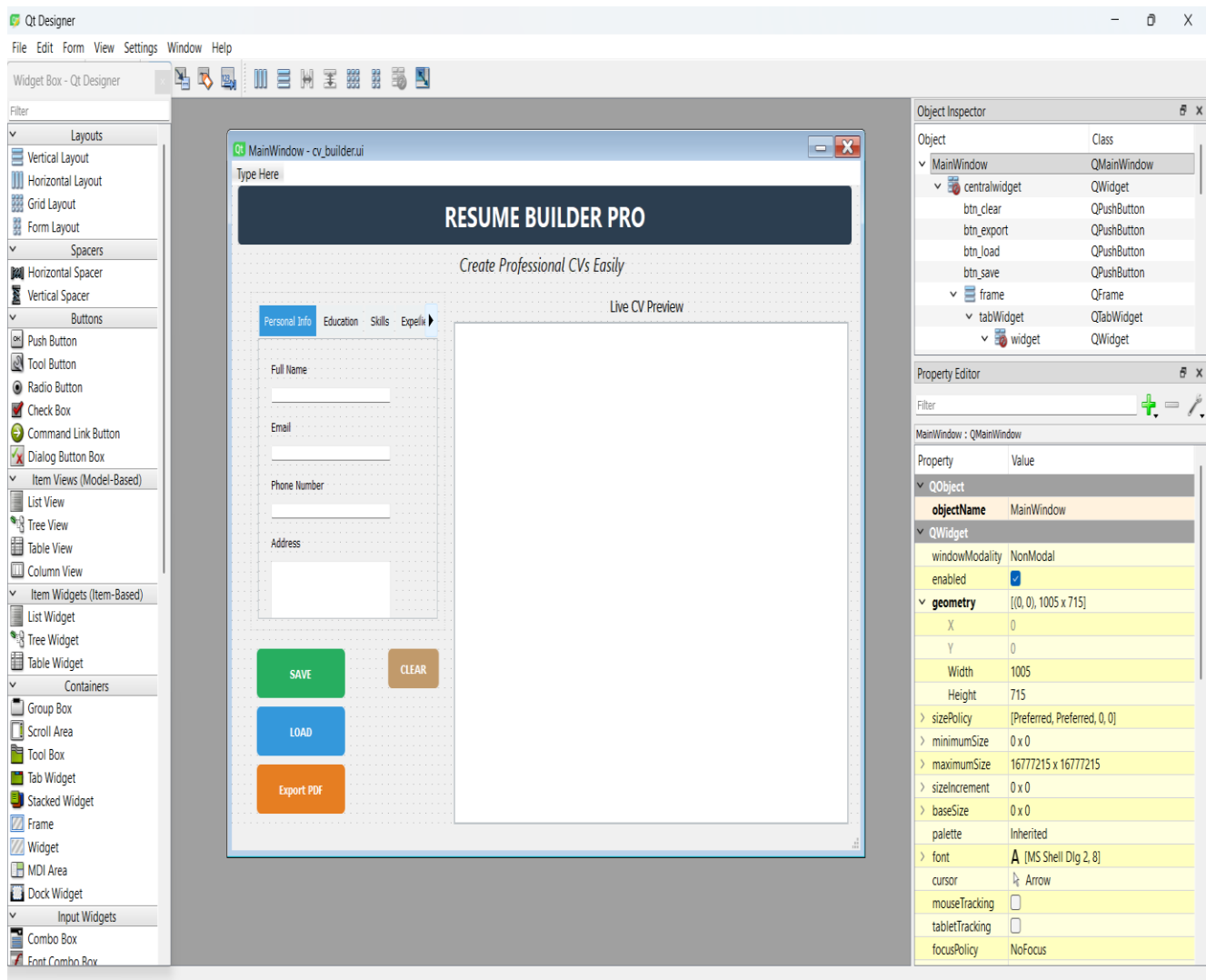
A Graphical User Interface (GUI) allows users to interact with software through visual components such as buttons, labels, text boxes, and menus. Qt Designer provides a drag-and-drop environment for designing interfaces, while Python is used to implement the application's functionality. GUI applications improve usability compared to command-line programs.

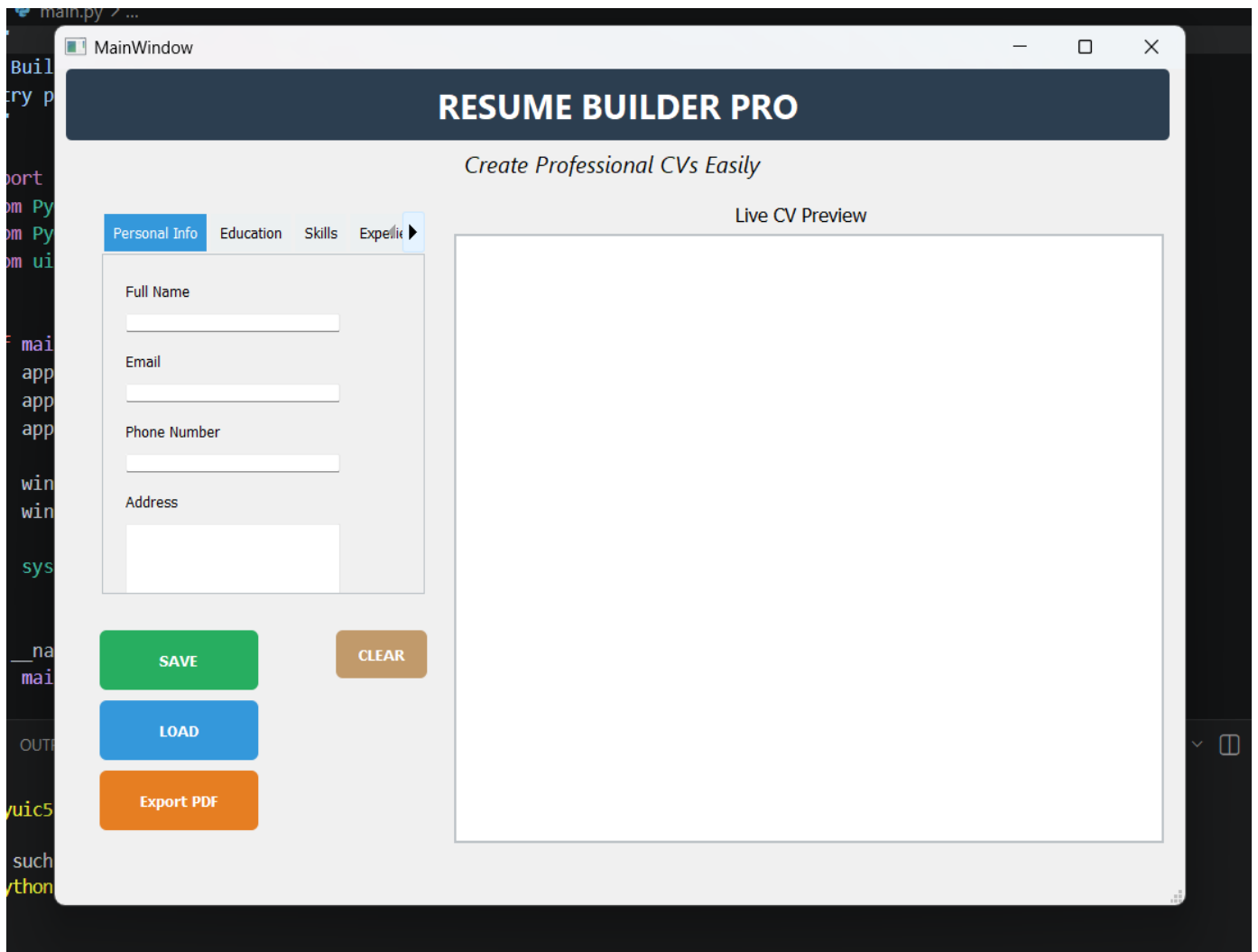
Lab Work:

Task 1:

Using Qt Designer to make a GUI.

Result:





Conclusions:

In this lab, graphical user interfaces were designed using Qt Designer and integrated with Python code. The lab demonstrated how GUI applications provide a more interactive and user-friendly experience compared to console-based programs.
